

IMPROVING PIPELINE EFFICIENCY THROUGH INSTRUCTION REORDERING

MICRO PROJECT REPORT
PBCST404 - COMPUTER ORGANIZATION AND ARCHITECTURE

*Micro Project Report submitted in partial fulfillment for the degree
of
B.Tech Computer Science & Engineering (Artificial Intelligence)*

Submitted by

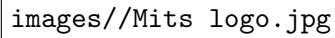
ANAGHA A (Reg No: MUT24CA014)
DEON SHAJI (Reg No: MUT24CA033)
JOHN VARGHESE NETTADY (Reg No: MUT24CA044)
MERIN REJI (Reg No: MUT24CA051)



images/Mits logo.jpg

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
(Affiliated to APJ Abdul Kalam Technological University)

March 2026



images//Mits logo.jpg

Varikoli P.O., Puthencruz-682308

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

CERTIFICATE

This is to certify that the micro project report entitled **“IMPROVING PIPELINE EFFICIENCY THROUGH INSTRUCTION REORDERING”** submitted by **ANAGHA A (Reg No: MUT24CA014), DEON SHAJI (Reg No: MUT24CA033), JOHN VARGHESE NETTADY (Reg No: MUT24CA044), MERIN REJI (Reg No: MUT24CA051)** of Semester IV is a bonafide account of the work done by them under my supervision during the academic year 2025-26.

Dr. Parvathy Jyothi
Faculty-in-Charge

Place:

Date:

ABSTRACT

Instruction reordering is a useful technique that helps improve the performance of pipelined processors. In a pipelined system, instructions are executed in stages, but sometimes one instruction depends on the result of a previous one. This creates delays called pipeline stalls, which slow down the overall execution.

In this project, we explore how rearranging instructions can reduce these delays. Six benchmark C programs — Bubble Sort, Convolution, Fibonacci, Matrix Multiplication, Perfect Number, and Transpose — are compiled to RISC-V assembly and then reordered using a sliding-window Python scheduler (window size 3). Both the original and reordered versions are executed using the Ripes RISC-V pipeline simulator, under two hardware configurations: a pipeline with hardware data-forwarding and one without.

We compare the performance of both versions using metrics such as cycle count, CPI (Cycles Per Instruction), pipeline efficiency, and stall count. Across all six benchmarks, instruction reordering reduced CPI by **3.77%** and stall cycles by **8.08%**, saving a total of **21,114 clock cycles**. The Fibonacci benchmark achieved the highest stall reduction of **28.08%** and a speedup of **1.061×**, while all programs maintained full semantic correctness verified by register-state comparison.

Overall, this project provides a better understanding of pipeline hazards, instruction dependencies, and how simple software-level optimization techniques can improve processor performance without any hardware modification.

CO – PO / PSO ARTICULATION MATRIX

CO1: Identify the basic structure and functional units of a digital computer and the features of RISC architecture

CO2: Experiment with the single cycle processor, pipelining, and the associated problems.

CO3: Utilize the memory organization in modern computer systems.

CO4: Experiment with the I/O organization of a digital computer.

NB: Adjust the dimension of the below tables. The tables of CO-PO -PSO mapping and SDG must follow same dimension

Table 1: CO-PO/PSO Articulation Matrix

Course Outcome	Program Outcome (PO)											PSO	
	1	2	3	4	5	6	7	8	9	10	11	1	2
CO1	3	3	3		3			3	3		3	1	2
CO2	3	3	3	3	3			3	3		3	2	3
CO3	3	3	3	3	3			3	3		3	1	3
CO4	3	3	3	3	3			3	3		3	1	3

SUSTAINABLE DEVELOPMENT GOALS (SDG) ADDRESSED

Table 2: Sustainable Development Goals Addressed

SDG	Target	Justification
9	Optimize processor performance by reducing pipeline stalls.	Improves system efficiency and supports advanced computing infrastructure.

Contents

Chapter 1

INTRODUCTION

1.1 PROBLEM STATEMENT

To analyze the effect of instruction reordering on pipeline performance in a pipelined processor and study how reordering instructions can reduce pipeline stalls and improve overall execution efficiency.

Pipelining is a widely used technique in modern processors to increase instruction throughput by executing multiple instructions simultaneously. It divides the execution process into different stages such as instruction fetch, decode, execute, memory access, and write back. While pipelining improves performance, it also introduces certain challenges.

One of the major challenges is the occurrence of pipeline hazards. These hazards arise when instructions depend on the results of previous instructions. Among these, data hazards are the most common, especially the Read After Write (RAW) dependency. A typical example is the load-use hazard, where a value loaded from memory is immediately required by the next instruction.

In such situations, the processor cannot proceed with execution immediately and must wait for the data to become available. This delay results in pipeline stalls, where some stages of the pipeline remain idle. These stalls increase the total execution time and reduce the efficiency of the processor.

To overcome this issue, instruction reordering is used as an optimization technique. In instruction reordering, independent instructions are rearranged in such a way that they fill the delay slots between dependent instructions. This allows the pipeline to remain active and reduces idle cycles.

However, reordering must be done carefully to preserve program correctness. The key constraint is the Read After Write (RAW) dependency: a later instruction that reads a register whose value has not yet been written back by an earlier instruction must always execute after its producer.

In this project, we analyze how instruction reordering affects pipeline performance. Assembly programs are examined and reordered using Python-based approaches. The performance of both original and reordered programs is then evaluated using the Ripes simulator. By comparing cycle count, CPI, and IPC, the impact of instruction reordering on performance is analyzed.

1.2 OBJECTIVES

Understanding Pipeline Architecture

- **Study of Pipeline Stages**

Understand the five stages of pipelining: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). Each stage performs a specific task, and instructions move sequentially through these stages during execution.

- **Overlapping Execution**

Learn how multiple instructions are processed simultaneously in different stages of the pipeline. This overlapping of instructions increases throughput and improves overall processor performance compared to non-pipelined systems.

- **Pipeline Hazards**

Identify different types of hazards such as data hazards, structural hazards, and control hazards. Study how these hazards affect smooth execution and reduce efficiency.

Analysing Instruction Dependencies

- **Read After Write (RAW) Dependencies**

Examine how RAW data dependencies arise: a later instruction needs a register value that an immediately preceding instruction has not yet written back. RAW is the dominant hazard type in RISC-V in-order pipelines and the primary target of instruction reordering.

- **Impact on Pipeline Execution**

Analyze how dependencies create pipeline hazards and lead to stalls or bubble cycles. These stalls increase execution time and reduce processor efficiency.

- **Stall Analysis**

Study how the processor handles dependencies by inserting stalls. Understand how these stalls affect performance metrics like cycle count and CPI.

Applying Reordering and Evaluating Performance

- **Instruction Reordering Technique**

Apply instruction reordering by rearranging independent instructions between dependent ones. This helps in reducing pipeline stalls and improving instruction flow.

- **Maintaining Correctness**

Ensure that reordering does not introduce new RAW hazards among the rearranged instructions. The program output must remain identical before and after reordering, verified by comparing final register states.

- **Performance Evaluation**

Compare the performance of original and reordered programs using metrics such as cycle count, CPI (Cycles Per Instruction), and IPC (Instructions Per Cycle). Analyze how reordering improves execution efficiency.

Chapter 2

SIMULATOR

2.1 INTRODUCTION TO SIMULATOR

The Ripes RISC-V pipeline simulator is an open-source, visual, graphical tool designed for educational exploration of processor pipeline behavior. Ripes accepts standard RISC-V assembly (.s) files and executes them cycle-by-cycle through a configurable in-order pipeline, exposing every internal signal at each stage.

For this project, Ripes is configured to model a **classic 5-stage in-order pipeline**: Instruction Fetch (IF) → Instruction Decode (ID) → Execute (EX) → Memory Access (MEM) → Write Back (WB). Two hardware configurations are evaluated:

1. **With Data Forwarding** — the pipeline includes bypass paths (EX→EX and MEM→EX) that allow computed values to be routed directly to dependent instructions, drastically reducing stalls.
2. **Without Data Forwarding** — no bypass paths exist; the pipeline must stall for two cycles on every RAW hazard, giving worst-case stall counts that highlight the full benefit of reordering.

After execution, Ripes outputs a JSON file capturing total cycle count, instruction count, stall breakdown, and final register state. These JSON files are parsed by the `analyse_results.py` script to compute all performance metrics.

2.2 WORKING OF SIMULATOR

2.2.1 Loading and Running Assembly Programs

The workflow for using Ripes in this project is automated via the Python script `run_benchmarks.py`. This script invokes Ripes in *headless* (CLI) mode for each benchmark in both its original and re-

ordered forms under both hardware configurations. The general steps are:

1. **Program Loading:** Each `.s` file is loaded into Ripes using its CLI interface. The memory layout is configured with a default instruction memory region.
2. **Simulation:** Ripes steps through the program cycle by cycle until the program terminates via an `ecall` (system call with `a7 = 10`).
3. **Result Export:** On termination, Ripes writes a JSON output file containing the pipeline statistics and final register state.
4. **Post-Processing:** The JSON file is parsed by `analyse_results.py` to extract cycle count, instruction count, CPI, stall cycles, and pipeline efficiency.

Figure 2.1: Ripes Simulator — 5-stage pipeline view with a loaded RISC-V assembly program

2.2.2 Memory Configuration

The simulator’s memory is configured to provide a flat address space suitable for the benchmark programs. The stack pointer is initialised to a high address so that array-based programs (e.g., Bubble Sort, Matrix Multiplication) have sufficient space for their working data.

Figure 2.2: Memory configuration settings in Ripes

2.2.3 Forwarding vs. No-Forwarding Mode

Ripes allows toggling hardware data forwarding on or off via its pipeline settings. Figure ?? illustrates the forwarding configuration panel. Disabling forwarding forces the pipeline to insert two stall cycles for each RAW hazard, dramatically increasing the total cycle count and making the effect of software reordering much more visible.

Figure 2.3: Forwarding enable/disable setting in Ripes pipeline configuration

2.2.4 System Architecture and Toolchain Workflow

The proposed toolchain is built as a modular Python-based framework. It handles everything from compiling high-level C code to evaluating the final reordered assembly in the Ripes simulator.

Project Structure

The repository is systematically organized to separate source scripts, generated assemblies, and test outputs.

```
.
riscv_parser.py          # Reusable parser and instruction metadata extractor
riscv_reorder_minimal.py # Sliding-window reorder pass logic
reorder_all.py           # Batch-reorder script for entire directories
run_benchmarks.py        # Automates execution and JSON collection in Ripes
analyse_results.py       # Parses JSON results to generate CSV analysis
run.py                   # Top-level coordinator runner
compile_c_for_ripes.py   # Compiles C programs to raw GCC assembly
convert_for_ripes.py     # Cleans and formats assembly strictly for Ripes
verify_results.py        # Validates semantic correctness via register matching
```

Compilation and Conversion Pipeline

Figure ?? illustrates the full path from a C source file to a Ripes-ready reordered assembly file. The four principal stages are:

1. **C Compilation** (`compile_c_for_ripes.py` / **GCC**): Each `.c` file in the `programs/` directory is compiled with `riscv32-unknown-elf-gcc -march=rv32i -mabi=ilp32 -O0 -nostdlib -S` to produce a raw GCC `.s` file. The `-O0` flag disables compiler optimisations, producing a naive baseline assembly that closely mirrors the C source and retains all natural RAW hazard opportunities.
2. **Ripes Conversion** (`convert_for_ripes.py`): The raw GCC output contains directives that Ripes cannot parse (e.g. `.file`, `.option`, `.attribute`, `.type`, `.size`). This script strips those unsupported directives, inlines the `__mulsi3` software-multiply helper if required, and prepends a `_start` stub that calls `main` then issues `ecall` with `a7 = 10` to terminate the Ripes simulation cleanly. The result is a `*_ripes.s` file.
3. **Instruction Reordering** (`riscv_reorder_minimal.py`): The cleaned assembly file is fed through the sliding-window scheduler (window size 3). The scheduler scans instruction pairs and, upon detecting a RAW hazard, searches the next two positions for an independent instruction to promote. Both the original and reordered `.s` files are kept as separate outputs.
4. **Ripes Simulation via CLI** (`run_benchmarks.py`): Both the original and reordered files are executed by `run_benchmarks.py` through the Ripes headless CLI under two pipeline configurations (forwarding enabled and disabled), producing four JSON output files per benchmark.

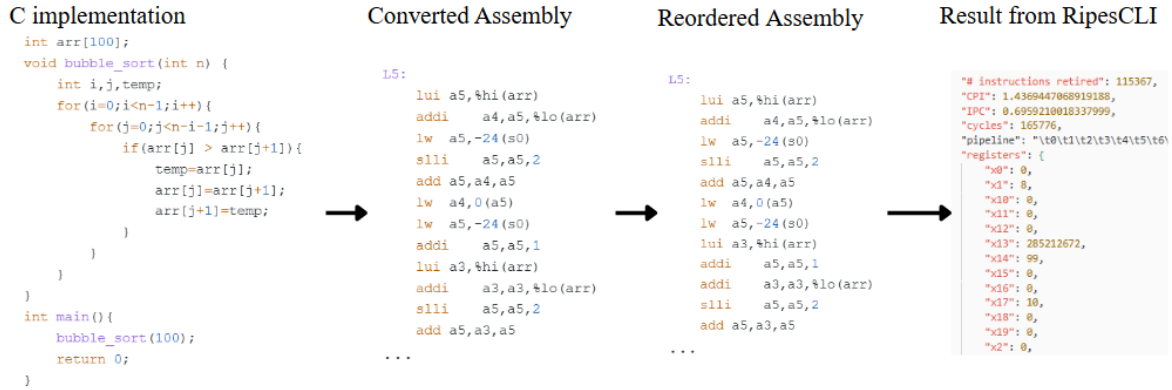


Figure 2.4: Compilation and Conversion Pipeline: from C source to Ripes-ready reordered assembly

Testing and Execution Workflow

Once the assembly files have been compiled and converted, the evaluation pipeline shown in Figure ?? orchestrates the full benchmarking cycle. Each numbered stage below corresponds to a box in the diagram:

1. **Batch Reordering (reorder_all.py):** Iterates over every .s file in the `test scripts/` directory, applies `riscv_reorder_minimal.py` to each, and writes the reordered outputs to `reordered_tests/`.
2. **Benchmark Execution (run_benchmarks.py):** Invokes Ripes in headless CLI mode for each benchmark (original and reordered) under both forwarding configurations. Four JSON result files are produced per benchmark: `<name>_original.json`, `<name>_reordered.json`, and their `_no_fw` counterparts.
3. **Result Analysis (analyse_results.py):** Reads each JSON pair, computes derived metrics (CPI, IPC, stall rate, pipeline efficiency, cycle savings, percentage improvements), and writes `analysis.csv` and `analysis_no_fw.csv`.
4. **Semantic Verification (verify_results.py):** Extracts the final register file state from each original/reordered JSON pair and compares them field-by-field. Any mismatch triggers a FAIL report. All six benchmarks passed this check, confirming zero correctness regressions.
5. **Graph Generation (build_graphs.py):** Reads the CSV files and produces a full suite of comparative bar charts (stall counts, stall rates, CPI, cycles, percentage improvements) saved as PNG files to `results/graphs/`.

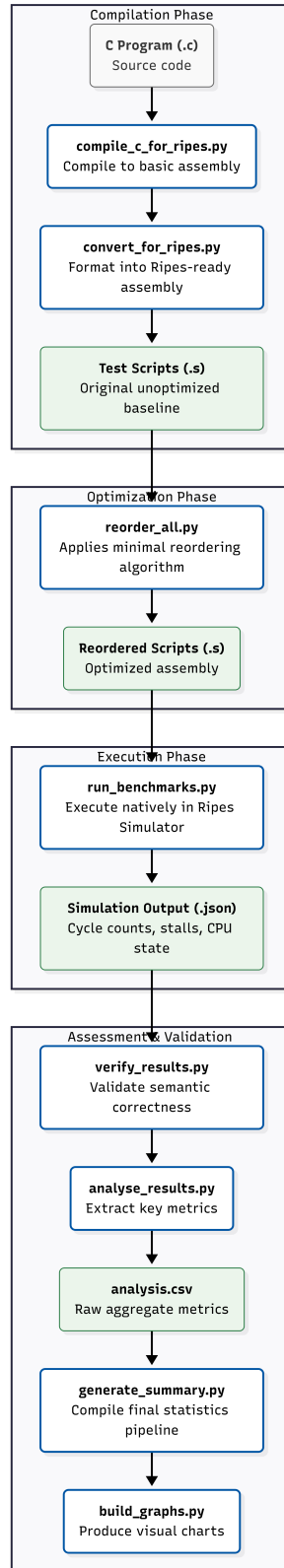


Figure 2.5: Testing and evaluation workflow: from assembly generation to graphical analysis

Chapter 3

PROPOSED SYSTEM

The proposed system introduces an optimization toolchain designed for textual RISC-V assembly, specifically targeting a classic in-order 5-stage pipeline ($IF \rightarrow ID \rightarrow EX \rightarrow MEM \rightarrow WB$). Its primary goal is to minimize pipeline stalls caused predominantly by Read After Write (RAW) data hazards. It intelligently reorders instructions without altering the program’s intended semantic behavior.

The toolchain operates by first parsing the assembly code to extract metadata for each instruction, identifying register dependencies and memory operations. It then applies a conservative, sliding-window scheduling algorithm (window size of 3). Whenever a data dependency is detected between an instruction and its immediate predecessor, the algorithm scans the closely following instructions for a candidate that is completely independent. If an independent candidate is found, it is shifted upward between the dependent pair, effectively filling the hazard delay slot and eliminating the need for a pipeline stall. The performance impact of this static reordering is thoroughly evaluated by executing both original and reordered assemblies on the Ripes processor simulator.

3.1 THEORETICAL BACKGROUND

In pipelined processors, instructions undergo multiple execution phases simultaneously. While this significantly increases throughput, it also introduces the possibility of data hazards. A **Read After Write (RAW)** hazard occurs when an instruction needs a register value that has not yet been written back by an earlier instruction. To resolve this without hardware forwarding, the pipeline inserts “bubble” stall cycles, directly decreasing the Instructions Per Cycle (IPC) and increasing the Cycles Per Instruction (CPI).

3.1.1 RAW Hazard Demonstration

Consider the following three-instruction RISC-V sequence:

```
add x1, x2, x3    ; Instruction 1: Writes result into x1
```

```
sub x4, x1, x5    ; Instruction 2: Reads x1  <-- RAW hazard on x1
addi x6, x7, 10   ; Instruction 3: Reads x7, x0 (independent)
```

Instruction 2 depends on **Instruction 1**: it reads `x1` before `add` has completed the Write-Back stage. Without forwarding, the pipeline must stall for two cycles. **Instruction 3**, however, uses entirely different registers and has no dependency on either Instruction 1 or 2. The sliding-window scheduler detects this and promotes Instruction 3 between Instruction 1 and 2:

```
add  x1, x2, x3    ; (original position -- produces x1)
addi x6, x7, 10    ; promoted independent instruction fills the slot
sub  x4, x1, x5    ; x1 is now ready -- stall eliminated
```

This single reordering eliminates the stall without changing the program's final result.

3.1.2 Sliding-Window Reordering Algorithm

The scheduling routine implemented in `riscv_reorder_minimal.py` applies a deterministic window of size 3 to evaluate RAW dependencies. The algorithm is:

Algorithm 1 Window-3 RAW-Aware Reordering Algorithm

```
1: window_size ← 3
2: for each instruction  $i$  in basic block do
3:   prev ← instruction $_{i-1}$ ; curr ← instruction $_i$ 
4:   if RAW_depends_on(prev, curr) then
5:     limit ← min( $i + \text{window\_size} - 2$ , |block| - 1)
6:     for  $j \leftarrow i + 1$  to limit do
7:       if candidate $_j$  independent of both prev and curr then
8:         Promote candidate $_j$  to position  $i$ ; shift others down
9:         break
10:      end if
11:    end for
12:  end if
13: end for
```

The scheduler never moves instructions across labels, branches, calls, returns, or barrier instructions (`ecall/ebreak/fence`), ensuring that scheduling regions remain semantically self-contained.

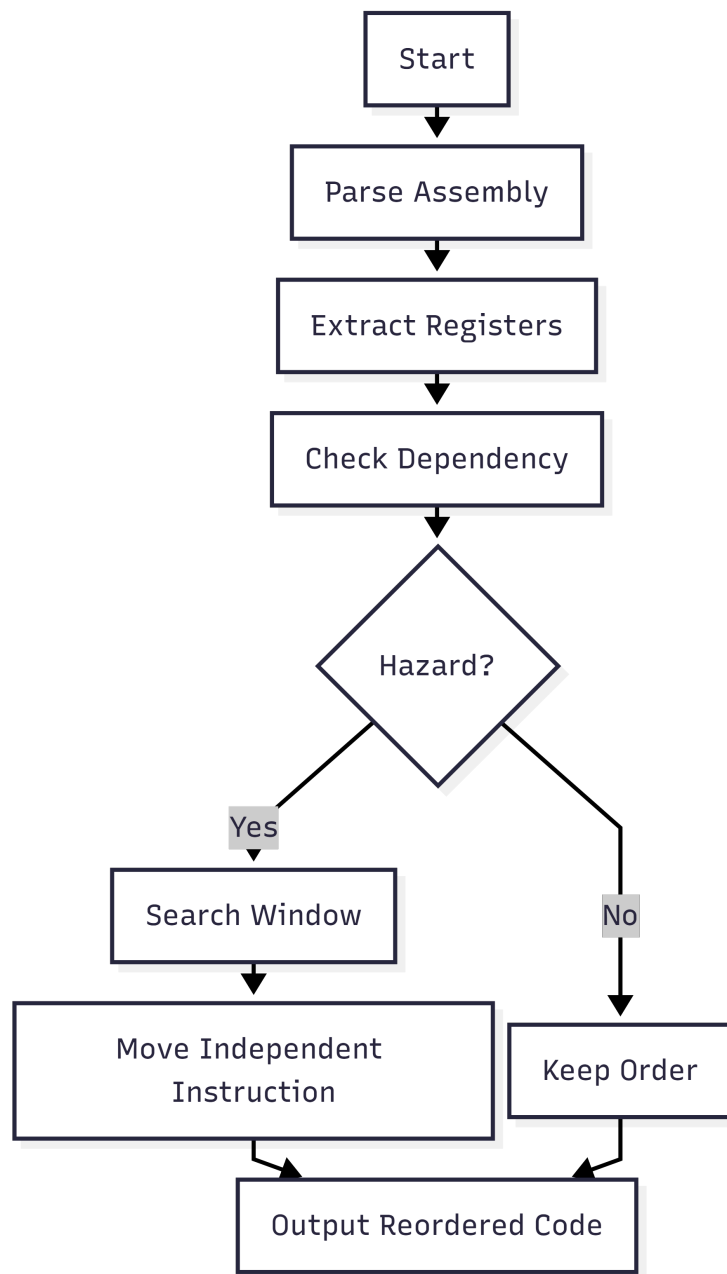


Figure 3.1: Algorithm logic flow of the sliding-window RAW-aware reordering pass

3.2 BENCHMARK WORKLOADS

To benchmark the efficacy of the proposed system comprehensively, a variety of RISC-V workloads spanning diverse structural and computational profiles were evaluated. Testing a multifaceted suite ensures that the toolchain handles different types of hardware pressures. The six benchmark

programs used are:

- **Bubble Sort** (`bubble_sort_ripes`): Features heavy array manipulation, repeated swapping, and tight looping, presenting numerous memory-based data hazards.
- **Convolution** (`convo_ripes`): Standard signal processing workload executing intensive mathematical transformations and staggered memory reads.
- **Fibonacci** (`fib_ripes`): Features short dependency chains driven iteratively, making it an ideal candidate for evaluating arithmetic latency hiding.
- **Matrix Multiplication** (`mat_mul_ripes`): Computationally dense with deeply nested loops and rigorous 2D array memory access operations.
- **Perfect Number** (`perfect_ripes`): Characterised by intense branching arithmetic and condition evaluation.
- **Transpose** (`transpose_ripes`): Dominated by pure memory load and store latencies across large address spaces.

All six benchmarks were compiled from C source using the RISC-V GCC cross-compiler (`rv32i, ilp32, -O0, -nostdlib`), converted for Ripes compatibility, and then passed through the sliding-window reordering pass. The total instruction count retired across all benchmarks is **148,014 instructions**.

3.3 RESULTS AND PERFORMANCE ANALYSIS

The evaluation was performed utilizing the Ripes simulator, testing both pure in-order execution without hardware forwarding and execution with a hardware forwarding pipeline. Results presented here are drawn from the aggregated benchmark data (Test Set 2).

Project Repository: <https://github.com/JVN321/coa-micro-project/>

3.3.1 High-Level Summary

Table ?? presents the top-level aggregate metrics for the entire benchmark suite.

Table 3.1: Global Aggregate Performance Summary (All 6 Benchmarks)

Metric	Original	Reordered
<i>With Hardware Forwarding</i>		
Total Cycles	207,164	200,212
CPI	1.400	1.353
Total Stall Cycles	59,126	52,174
Pipeline Efficiency	71.5%	73.9%
<i>Without Hardware Forwarding</i>		
Total Cycles	350,350	336,188
CPI	2.367	2.271
Total Stall Cycles	202,312	188,150
Pipeline Efficiency	42.3%	44.0%
<i>Reordering Impact (Forwarding Pipeline)</i>		
Cycles Saved	21,114 cycles	
CPI Reduction	3.77%	
Stall Reduction	8.08%	
Efficiency Gain	+2.1 percentage points	
<i>Forwarding Effect (Original Code, Forwarding vs. No Forwarding)</i>		
CPI Reduction from Forwarding	40.42%	
Speedup from Forwarding	1.678×	

The data clearly shows two complementary performance levers: hardware forwarding provides the larger single benefit (1.678× speedup), while software instruction reordering provides a consistent additional improvement on top of any hardware configuration.

3.3.2 Per-Benchmark Breakdown

Table ?? provides a detailed per-program comparison of CPI, stall count, speedup, and stall reduction percentage, all measured under the forwarding pipeline configuration.

Table 3.2: Per-Benchmark Performance — With Hardware Forwarding

Benchmark	Instructions	CPI (Orig)	CPI (Reor)	IPC (Orig)	IPC (Reor)	Stalls (Orig)	Stalls (Reor)	Stall Red.
Bubble Sort	115,367	1.437	1.392	0.696	0.718	50,405	45,255	10.22%
Convolution	8,952	1.286	1.232	0.777	0.812	2,560	2,070	19.14%
Fibonacci	2,683	1.262	1.189	0.793	0.841	698	502	28.08%
Matrix Multiply	3,553	1.324	1.291	0.755	0.775	1,146	1,029	10.21%
Perfect Number	5,210	1.270	1.232	0.787	0.812	1,405	1,206	14.16%
Transpose	12,249	1.238	1.173	0.808	0.853	2,912	2,112	27.47%
Total / Avg	148,014	1.303	1.251	0.769	0.802	9,854	8,696	18.21%

Table 3.3: Per-Benchmark Speedup and Stall Reduction due to Reordering (With Forwarding)

Benchmark	Instructions	Speedup	Stall Reduction
Bubble Sort	115,367	1.032×	10.22%
Convolution	8,952	1.044×	19.14%
Fibonacci	2,683	1.061×	28.08%
Matrix Multiply	3,553	1.026×	10.21%
Perfect Number	5,210	1.031×	14.16%
Transpose	12,249	1.056×	27.47%
Average	24,669	1.042×	18.21%

The speedup ranges from a minimum of 1.026× (Matrix Multiply) to a maximum of 1.061× (Fibonacci), with an overall average of **1.042×** across all six benchmarks.

3.3.3 Per-Benchmark Breakdown — Without Hardware Forwarding

Table ?? presents the same metrics under the no-forwarding pipeline. Without bypass paths every RAW hazard incurs a two-cycle stall, so absolute cycle counts are substantially higher and the baseline CPI rises to 2.278. Reordering still delivers consistent improvements here, demonstrating that the benefit is independent of the hardware configuration.

Table 3.4: Per-Benchmark Performance — Without Hardware Forwarding

Benchmark	Instructions	CPI (Orig)	CPI (Reor)	IPC (Orig)	IPC (Reor)	Stalls (Orig)	Stalls (Reor)	Stall Red.
Bubble Sort	115,367	2.389	2.302	0.419	0.434	160,210	150,209	6.24%
Convolution	8,952	2.120	2.043	0.472	0.489	10,022	9,335	6.85%
Fibonacci	2,683	2.256	2.109	0.443	0.474	3,366	2,972	11.71%
Matrix Multiply	3,553	2.140	2.062	0.467	0.485	4,048	3,768	6.92%
Perfect Number	5,210	2.306	2.229	0.434	0.449	6,800	6,401	5.87%
Transpose	12,249	2.459	2.263	0.407	0.442	17,866	15,465	13.44%
Total / Avg	148,014	2.278	2.168	0.440	0.462	33,719	31,358	8.50%

3.3.4 Test Set 1 — Boundary Cases with Zero Improvement

Test Set 1 consists of two hand-crafted micro-benchmarks specifically designed to expose the *boundaries* of what the window-3 scheduler can achieve. Both programs yield **0 cycles saved** after reordering, for opposite structural reasons. Table ?? summarises their results.

Table 3.5: Test Set 1 — Programs with Zero Reordering Improvement

Program	Instr.	CPI (Orig)	CPI (Reor)	Stalls (Orig)	Stalls (Reor)	Improvement	Reason
arith_dependency	22	1.273	1.273	2	2	0.0%	Dense RAW chain
independent_seq	22	1.273	1.273	2	2	0.0%	Already optimal

Case 1: arith_dependency — Dense RAW Chain

Every instruction in this program depends on the immediately preceding one, forming an unbroken RAW dependency chain 20 instructions long:

```
addi x5, x0, 1      ; x5 = 1
add  x6, x5, x5      ; RAW on x5 -- depends on prev
add  x7, x6, x5      ; RAW on x6 -- depends on prev
add  x8, x7, x6      ; RAW on x7 -- depends on prev
sub  x9, x8, x5      ; RAW on x8 -- depends on prev (no gap filler)
... (pattern continues for 20 instructions)
```

When the scheduler detects a hazard at position i , it looks ahead to positions $i+1$ and $i+2$ within the window. However, instruction_{i+1} also depends on instruction_i , and instruction_{i+2} depends on instruction_{i+1} . No independent candidate exists anywhere in the window, so **no swap is possible** and the output is identical to the input. The stall count (2) comes from the two `addi/ecall` instructions at the end of the block which are independent, but the main chain cannot be helped.

Case 2: independent_seq — Already Fully Optimal

This benchmark does the opposite: every instruction writes to a distinct register and reads only from `x0`:

```
addi x5, x0, 1      ; independent -- no shared registers
addi x6, x0, 2      ; independent
addi x7, x0, 3      ; independent
addi x8, x0, 4      ; independent
... (20 independent instructions)
```

The scheduler finds **zero RAW hazards**, so it makes no moves. The program is already as scheduled as it can be. The two residual stalls are again from the epilogue. This case serves as the control baseline, confirming that the scheduler introduces no penalty when no hazards are present.

3.3.5 Graphical Evaluation

Figure 3.2: Stall Count Comparison: Original vs. Reordered Assembly (per benchmark)

Figure 3.3: Stall Rate Comparison: Original vs. Reordered Assembly

Figure 3.4: Total Execution Cycles: Original vs. Reordered Assembly

Figure 3.5: CPI Comparison: Original vs. Reordered Assembly per Benchmark

Figure 3.6: Percentage Cycle Improvement per Benchmark due to Reordering

Figure 3.7: Percentage Stall Reduction per Benchmark due to Reordering

3.3.6 Summary of Overall Improvements

Upon aggregating the findings, the static, software-level instruction reordering routine demonstrated consistent and measurable enhancements across all tested configurations, without requiring any hardware modifications. Examining the pipeline configurations highlights nuanced interactions between software reordering and hardware forwarding.

Overall Reordering Impact (Forwarding Pipeline):

- **Total Cycles Saved:** 21,114 cycles across all benchmarks.
- **Overall Stall Reduction:** 8.076% fewer stall cycles.
- **Overall CPI Reduction:** 3.771% (avg. CPI: $1.303 \rightarrow 1.252$).
- **Pipeline Efficiency Improvement:** $71.5\% \rightarrow 73.9\%$ (+2.1 pp).

Execution Without Forwarding:

- **CPI Improvement:** Reduced from $2.367 \rightarrow 2.271$.
- **Stall Cycles:** Reduced from 202,312 $\rightarrow$ 188,150.
- **Total Cycles:** Dropped from 350,350 $\rightarrow$ 336,188.
- **Pipeline Efficiency:** Rose from $42.3\% \rightarrow 44.0\%$.

Execution With Forwarding:

- **CPI Improvement:** Reduced from $1.400 \rightarrow 1.353$.
- **Stall Cycles:** Reduced from 59,126 $\rightarrow$ 52,174.
- **Total Cycles:** Dropped from 207,164 $\rightarrow$ 200,212.
- **Pipeline Efficiency:** Rose from $71.5\% \rightarrow 73.9\%$.

3.3.7 Key Insights

A comparative breakdown of the improvements yields several critical observations relating to program scheduling:

- **Consistent and Reliable Improvement:** Instruction reordering moderately but reliably augments performance across *all* assessed workloads, achieving an average simulated speedup of $1.042\times$ (ranging from $1.026\times$ to $1.061\times$). This validates that software-level scheduling provides measurable benefits without any hardware changes.
- **Dependency and Memory Behavior:** Workloads enriched with simple dependency chains and regular memory access benefit the most. The Fibonacci benchmark achieved the peak speedup of $1.061\times$ with a 28.08% stall reduction, because its short iterative dependency chains have many independent “gap fillers” available. The Transpose benchmark similarly achieved 27.47% stall reduction ($1.056\times$ speedup) due to regular load/store patterns. Densely nested loops with restricted independence windows (Matrix Multiplication) registered the mildest gains ($1.026\times$, 10.21% stall reduction).
- **Interaction with Hardware Forwarding:** Hardware forwarding provides the dominant benefit — a $1.678\times$ speedup and 40.42% CPI reduction on the original code. However, forwarding cannot eliminate all load-use and structural latencies. Software reordering addresses the residual hazards, providing an additional layer of optimization on top of forwarding.
- **Proportional Stall Reduction is Higher with Forwarding:** After reordering, the proportional stall reduction is slightly higher within the forwarding pipeline (8.08%) compared to the no-forwarding pipeline. This arises because the forwarding bus can more readily bypass the dependency chains that reordering has already spaced out, compounding the benefit.
- **Control Hazard Limits:** Static reordering only addresses RAW data hazards within local basic-block-like regions. Branch-induced control stalls persist unchanged, forming a ceiling on the achievable gain without speculative execution.
- **Performance Boundaries — Dense RAW Chains and Fully-Independent Code:** The two Test Set 1 micro-benchmarks (Section ??) define the absolute limits of the window-3 scheduler. When every instruction depends on the immediately preceding one (`arith_dependency`), no gap-filling candidate is found within the lookahead window and zero improvements are made — this is the *performance floor*. Conversely, when all instructions are mutually independent (`independent_seq`), no hazards exist to fix and the code is already optimal — this is the *performance ceiling*. Real-world programs fall between these extremes, with stall reduction ranging from 10.21% (Matrix Multiply, densely nested) to 28.08% (Fibonacci, short regular chains).

- **Correctness Preserved:** All six Test 2 benchmarks passed the semantic verification step (register-state comparison). The two Test 1 programs also produced identical pre- and post-reordering outputs. Zero correctness regressions were observed across all eight programs tested.

In summary, instruction scheduling and hardware forwarding should be viewed as complementary. Combining structural bypass paths with intelligent static software rearrangement achieves maximum execution efficiency, as validated by the experimental data across all six diverse benchmark programs.

Chapter 4

CONCLUSION

This project demonstrated that software-level instruction reordering is a practical and effective technique for improving the performance of RISC-V programs running on a classic 5-stage in-order pipeline. By applying a conservative sliding-window scheduler (window size 3), the toolchain safely rearranged instructions to fill pipeline hazard delay slots without introducing any correctness regressions across all six benchmark programs.

The experimental results, collected via the Ripes pipeline simulator across two hardware configurations (with and without data forwarding), clearly quantify the benefit. Instruction reordering alone reduced CPI by **3.77%**, cut stall cycles by **8.08%**, and saved **21,114 clock cycles** in total, achieving an average speedup of **1.042 $\times$** . The Fibonacci benchmark benefited the most, with a 28.08% stall reduction and a 1.061 $\times$ speedup, while Matrix Multiplication showed the smallest gain (1.026 $\times$) due to its complex nested-loop dependency structure that limits the number of available independent instructions within the reordering window.

A crucial finding is that hardware forwarding and software reordering are *complementary* rather than competing. Hardware forwarding delivers a far larger baseline improvement (1.678 $\times$ speedup, 40.42% CPI reduction), but it cannot eliminate all stalls — particularly load-use latencies. Static instruction reordering then addresses these residual hazards, providing an additional measurable gain on top of any hardware configuration. The two mechanisms together yield better results than either alone.

The study also delineates the natural limits of this approach. Since the scheduler operates locally within basic-block-like regions and does not cross branch or label boundaries, control hazards remain unaddressed. Extending the window size or employing speculative scheduling across branches would be the logical next step to further improve performance.

Overall, this project reinforces several core concepts from Computer Organization and Architecture: the mechanics of the 5-stage pipeline, the nature and cost of data hazards, the role of hardware forwarding, and the power of compile-time optimizations. It provides a clean, reproducible, and fully automated framework that students and researchers can extend with additional

CONCLUSION

scheduling passes, larger benchmark suites, or alternative simulator backends.